



Introduzione alla virtualizzazione

Dai mainframe IBM al cloud

Prof. Michele Ciavotta

michele.ciavotta@unimib.it

Indice

- Introduzione
- Cos'è un hypervisor?
- Vantaggi
- Tecniche di virtualizzazione

Richiamo - Tipi di Mobilità

Weak Mobility (Mobilità Debole)



Weak mobility

Si trasferisce solo il **codice** (ed eventuali dati di inizializzazione); l'esecuzione riparte da zero sulla macchina di destinazione.

Esempio: applet Java, script scaricati dal browser.

Strong Mobility (Mobilità Forte)



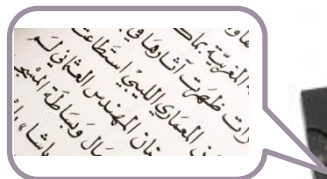
Strong mobility

Si trasferiscono codice **e segmento di esecuzione**: il processo riprende sulla destinazione esattamente nello stato in cui era stato sospeso.

Esempio: process migration, live migration di VM.

VIRTUALIZZAZIONE E MACCHINE VIRTUALI

Emulazione



あいうえお
かきくけこ



L'**emulazione** è una tecnica con cui un sistema (l'**host**) riproduce in software l'**hardware** e l'**instruction set** di un sistema diverso (il **guest**).

Un **emulatore** traduce le istruzioni dell'architettura guest in istruzioni che l'host è in grado di eseguire, ricostruendo via software il comportamento completo del sistema originale.

Virtualizzazione e macchine virtuali

La **virtualizzazione** è una tecnologia che crea **istanze logiche e astratte delle risorse fisiche di calcolo**.

Si avvale di un **hypervisor** (o Virtual Machine Monitor) per partizionare l'hardware fisico e creare più ambienti virtuali isolati, le **macchine virtuali (VM)**.

Ogni VM esegue il **proprio sistema operativo** e le proprie applicazioni nella convinzione di disporre di hardware dedicato, pur condividendo le risorse fisiche sottostanti.



Aspetto

Architettura

Prestazioni

Traduzione

Caso d'uso

Virtualizzazione

Stessa ISA (es. x86 su x86)

Vicine a quelle native

Minima o assente

Consolidamento server, cloud computing

Emulazione

ISA diversa (es. ARM su x86)

Significativamente inferiori

Traduzione completa delle istruzioni

Conservazione di sistemi legacy, test cross-platform

Virtualizzazione e macchine virtuali

Macchina virtuale: fornisce una *piattaforma hardware emulata completa*, in grado di ospitare un sistema operativo a tutti gli effetti.

- Le risorse della macchina fisica sottostante possono essere condivise tra più macchine virtuali, ciascuna con un proprio sistema operativo.
- Create e gestite da un **hypervisor** (detto anche **Virtual Machine Monitor**).
- Esempi: VirtualBox, KVM, VMware, Xen.



PROPRIETÀ 1

Equivalenza

Un programma in esecuzione nella VM mostra un comportamento *essenzialmente identico* a quello osservabile sulla macchina fisica, eccezion fatta per gli effetti di temporizzazione e di disponibilità delle risorse.

PROPRIETÀ 2

Controllo delle risorse

Il VMM mantiene il *controllo completo* delle risorse fisiche: nessun programma guest può accedere a risorse non esplicitamente allocate, né alterare l'allocazione decisa dal VMM.

PROPRIETÀ 3

Efficienza

Una *frazione statisticamente dominante* delle istruzioni macchina è eseguita direttamente dal processore fisico, senza intervento del VMM.

Sessant'anni di virtualizzazione

Breve cronologia:

- **1965** – IBM introduce la virtualizzazione con il CP-40 sui mainframe, consentendo l'esecuzione di più macchine virtuali su un unico sistema.
- **1972** - *VM/370 primo hypervisor commerciale*
- **1985** – AT&T 6300+ (Olivetti) con il VMM Simultask consente l'esecuzione concorrente di DOS e Unix SVR2: uno dei primi esperimenti di virtualizzazione su PC.
- **Anni '90–2000** – La virtualizzazione torna sui PC (VMware, VirtualBox) e diventa la base del cloud computing.
- **Oggi** – È adottata su larga scala nei data center, dove abilita la gestione unificata dell'hardware e la condivisione efficiente delle risorse.



Process VM e System VM a confronto



Una **process virtual machine** (o **application VM**) fornisce un ambiente di esecuzione indipendente dalla piattaforma per una singola applicazione.

A differenza delle system VM, che virtualizzano l'intero hardware, virtualizzano solo ciò che serve a quella applicazione.

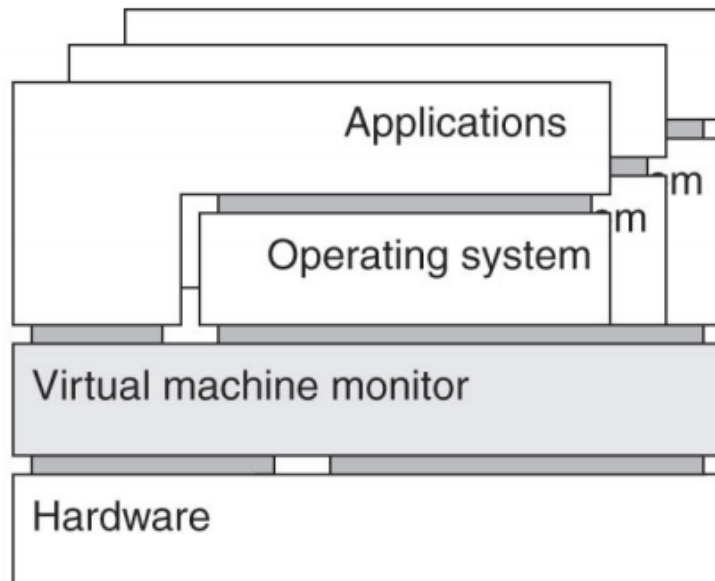
- **Singola applicazione:** ogni istanza esegue un solo processo.
- **Indipendenza dalla piattaforma:** «write once, run anywhere», ovunque sia disponibile la VM.
- **Esecuzione gestita:** memoria, garbage collection e sicurezza forniti dal runtime.
- **Rappresentazione intermedia:** il codice è compilato in bytecode, interpretato o compilato just-in-time dalla VM.

Cos'è un hypervisor?

Un **hypervisor** (o **Virtual Machine Monitor**) è un componente software che crea e gestisce le macchine virtuali astraendo e partizionando le risorse hardware fisiche.

Sfrutta le **estensioni hardware per la virtualizzazione** (come Intel VT-x o AMD-V) offerte dalle CPU moderne per gestire le VM in modo efficiente.

Condivide le risorse disponibili in modo che ciascuna VM le percepisca come **dedicate**.



The hardware is completely hidden by the VMM

Quali risorse?

- CPU
- Memoria
- Storage
- Rete

Classificazione delle istruzioni

Le istruzioni si classificano in base al **privilegio** richiesto e la **sensibilità** verso lo stato della macchina.

Asse privilegio:

- Le **privilegiate** generano una *trap* se eseguite in user mode;
- le **non privilegiate** vengono eseguite direttamente dalla CPU.

Asse sensibilità:

- Le **sensibili** influiscono sull' hardware condiviso fra le VM: accedono o modificano registri di sistema, memoria della macchina, interrupt.
- Le **innocue** restano dentro la VM: calcoli, salti, lettura e scrittura della propria memoria.



Matrice di virtualizzabilità

	PRIVILEGIATE	NON PRIVILEGIATE
<i>Sensibili</i>	Caso virtualizzabile. <i>Trap-and-emulate</i> applicabile.	Caso patologico. Richiede BT, paravirt o HW assist.
<i>Innocue</i>	Esecuzione diretta in user mode (ammessa, ma rara come categoria).	Esecuzione diretta — il caso normale, dominante in frequenza.

Teorema di Popek–Goldberg: l'architettura è virtualizzabile in senso classico quando la cella sensibili + non privilegiate è vuota.

PowerPC, SPARC v9, IBM S/390 la soddisfano; x86 a 32 bit no.

Virtualizzazione della CPU e protection ring

Le CPU moderne definiscono livelli di protezione (ring) che vanno da Ring 0 (massimo privilegio) a Ring 3 (privilegio minimo).

I kernel dei sistemi operativi sono progettati per girare in Ring 0 ed eseguire le istruzioni privilegiate direttamente sull'hardware.

L'hypervisor deve intercettare queste istruzioni: **trap-and-emulate**.

In un ambiente virtualizzato il guest OS crede di avere il pieno controllo dell'hardware, ma in realtà gira in **modalità deprivilegiata** (tipicamente in user mode o in un ring meno privilegiato).

Quando il guest OS tenta di eseguire un'istruzione privilegiata, la CPU genera un **trap** (un'eccezione) che trasferisce il controllo all'hypervisor, il quale **emula** il comportamento dell'istruzione in modo sicuro e virtualizzato.

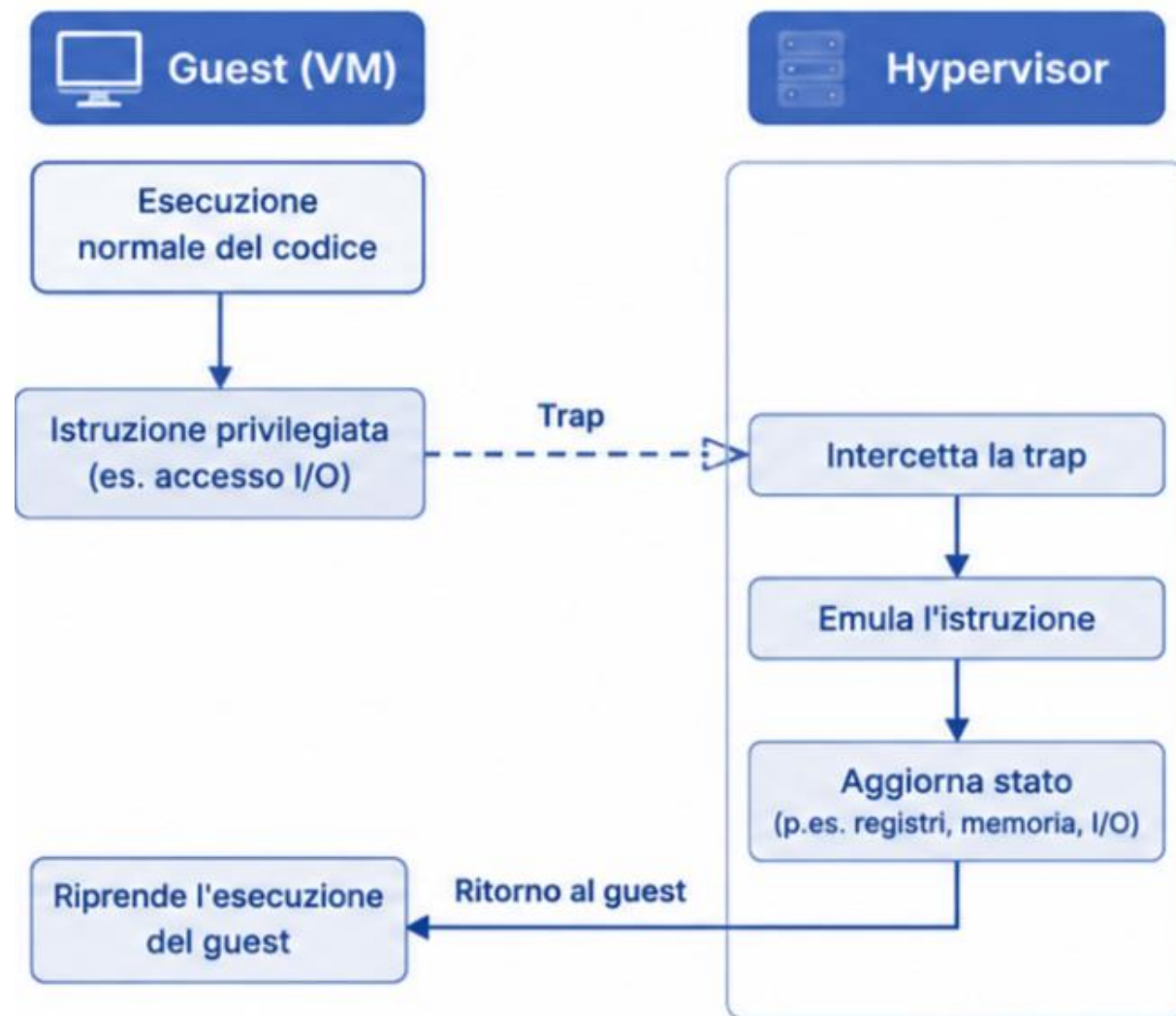
Trap-and-emulate

Obiettivo — permettere a un sistema operativo guest di eseguire codice come se avesse il controllo dell'hardware fisico.

Esecuzione normale — le istruzioni non privilegiate del guest vengono eseguite direttamente dalla CPU, senza intervento continuo dell'hypervisor.

Trap — quando il guest esegue un'istruzione privilegiata o accede a una risorsa protetta, la CPU genera una trap e trasferisce il controllo all'hypervisor.

Emulate — l'hypervisor interpreta la richiesta, aggiorna lo stato virtuale della VM ed emula l'effetto dell'operazione sull'hardware virtuale.



Virtualizzazione della CPU e protection ring

Le CPU moderne definiscono livelli di protezione (ring) che vanno da Ring 0 (massimo privilegio) a Ring 3 (privilegio minimo).

I kernel dei sistemi operativi sono progettati per girare in Ring 0 ed eseguire le istruzioni privilegiate direttamente sull'hardware.

L'hypervisor deve intercettare queste istruzioni: **trap-and-emulate**.

Per le istruzioni sensibili: **Traduzione binaria, Paravirtualizzazione, virtualizzazione assistita dall'hardware**

La traduzione binaria è una tecnica di virtualizzazione che, a tempo di esecuzione, scansiona, analizza e riscrive dinamicamente il codice del guest, sostituendo le istruzioni problematiche con equivalenti sicuri eseguibili in un ambiente virtualizzato.

È la soluzione adottata da VMware per virtualizzare l'architettura x86 prima dell'introduzione del supporto hardware alla virtualizzazione.

Traduzione binaria dinamica

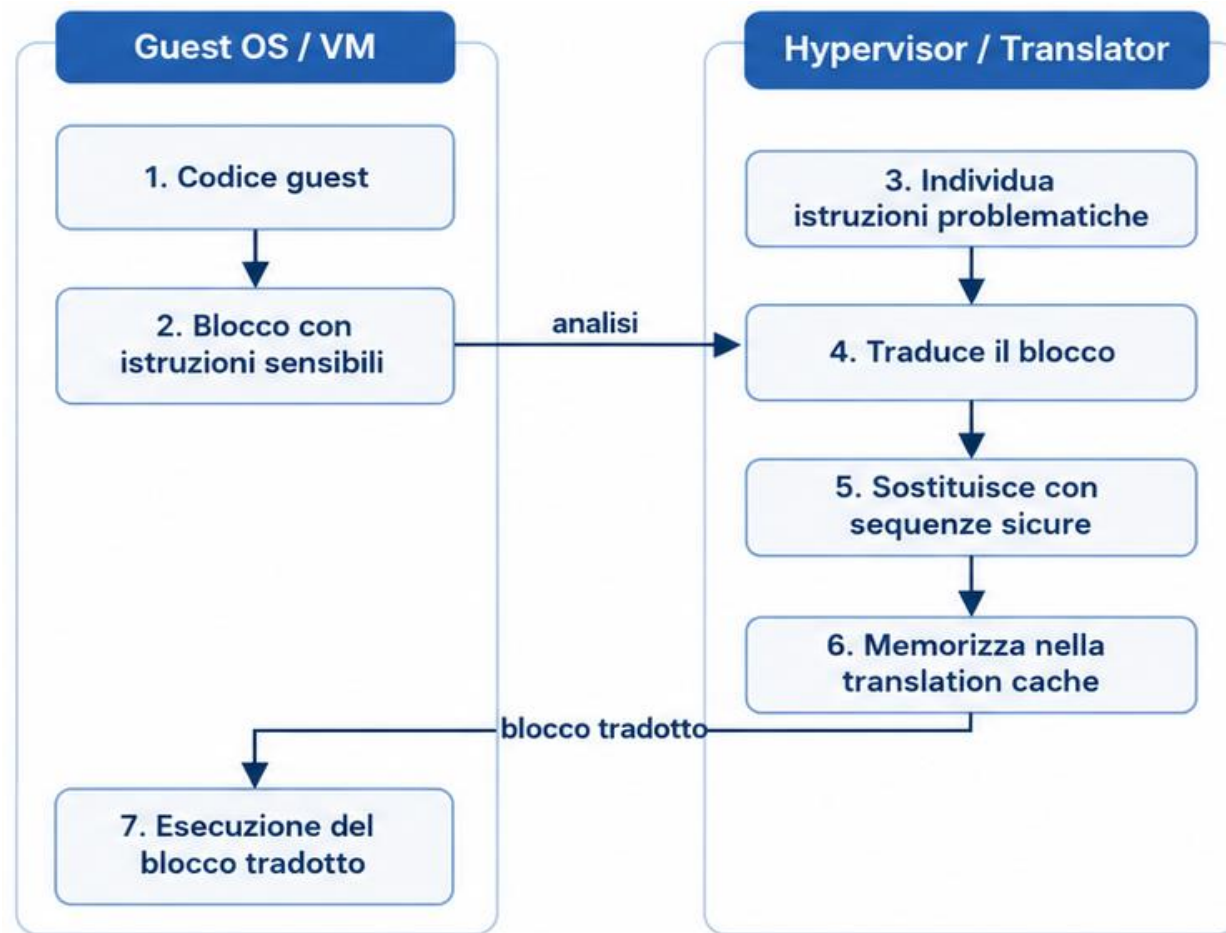
Obiettivo — eseguire un sistema guest quando alcune istruzioni sensibili non sono virtualizzabili con il solo trap-and-emulate.

Idea di base — l'hypervisor analizza il codice guest e riscrive i blocchi con istruzioni problematiche, sostituendoli con sequenze sicure.

Flusso — il guest esegue codice normale; sui blocchi sensibili il traduttore produce una versione equivalente sicura, che viene eseguita al loro posto.

Vantaggio — virtualizza architetture come lo x86 classico, dove alcune istruzioni sensibili non generano trap in modo affidabile.

Costo — più complesso del trap-and-emulate: overhead per traduzione, cache dei blocchi e gestione del controllo.



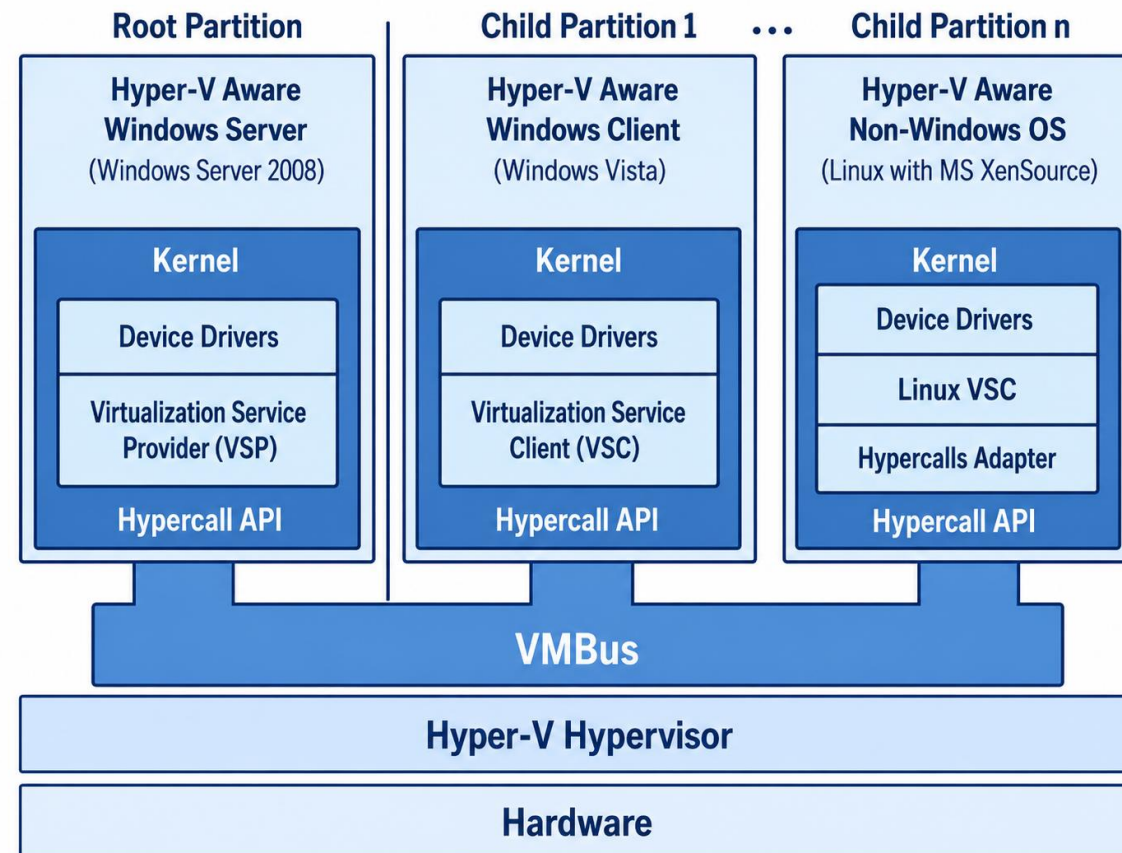
Paravirtualizzazione

La paravirtualizzazione è una tecnica di virtualizzazione in cui il **sistema operativo guest viene modificato** per essere consapevole di girare in un ambiente virtualizzato.

il guest usa **chiamate speciali** per interagire direttamente con l'hypervisor. Queste chiamate prendono il nome di **hypercall**.

i sistemi operativi guest sono consapevoli dell'hypervisor e utilizzano chiamate speciali per interagire con esso.

- Questo si traduce in:
 - migliori prestazioni
 - integrazione fluida (mouse, tastiera, schermo)
 - sicurezza ridotta



Virtualizzazione della CPU e anelli di protezione

Virtualizzazione hardware-assisted (Intel VT-x, AMD-V): la CPU aggiunge una **nuova dimensione di privilegio** indipendente dai ring (root vs non-root), così il kernel del guest può girare nel suo Ring 0 senza compromettere l'isolamento.

Due modalità di esecuzione

Root mode — esegue l'hypervisor, con pieno accesso all'hardware reale.

Non-root mode — esegue il guest, che usa normalmente i ring x86 (kernel in Ring 0, applicazioni in Ring 3). Le operazioni che richiedono l'intervento dell'hypervisor sono però intercettate dalla CPU.

VM-exit

Transizione hardware da non-root a root: la CPU sospende il guest e cede il controllo all'hypervisor. Può essere causata da istruzioni privilegiate, accessi I/O, eventi sulla memoria, interrupt, timer, o politiche configurate dall'hypervisor. Il trapping avviene in hardware; la gestione dell'evento resta software.

Estensioni correlate: EPT / NPT per la memoria, VT-d / AMD IOMMU per l'I/O.

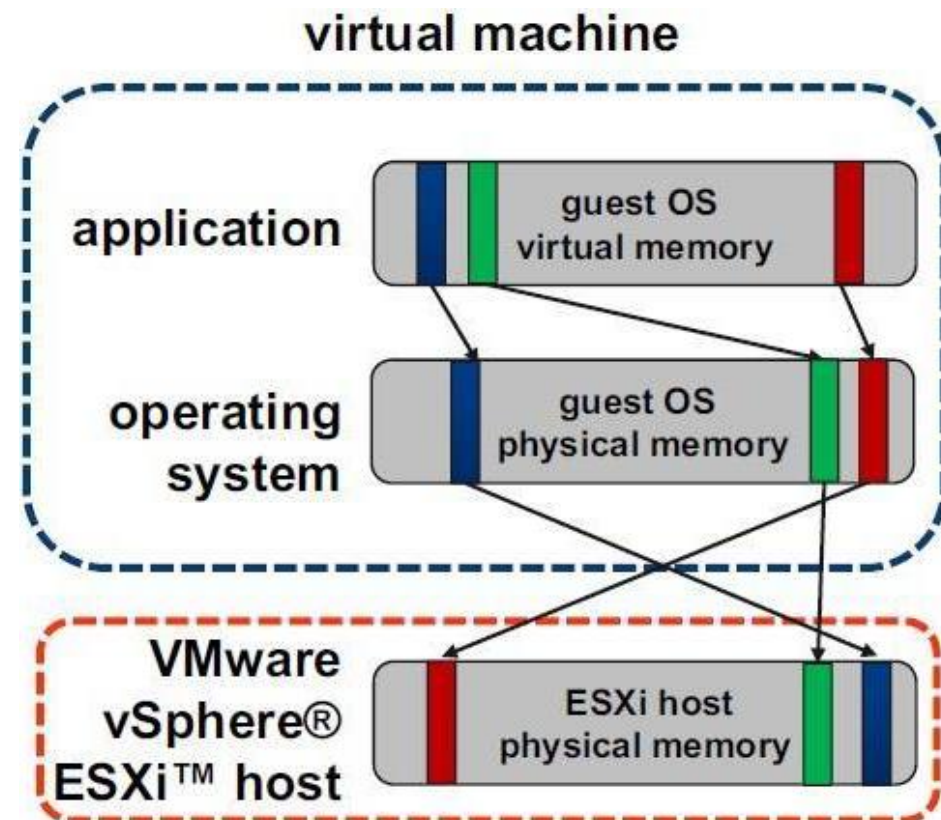
Virtualizzazione della memoria

In un sistema non virtualizzato l'OS gestisce un'unica traduzione: *virtuale* → *fisico* .

La virtualizzazione introduce un terzo spazio di indirizzamento e raddoppia il problema.

La gestione della memoria è una delle sfide cruciali della virtualizzazione: il guest OS mantiene le proprie page table assumendo di possedere la memoria fisica.

Shadow page table: l'hypervisor mantiene copie sincronizzate delle page table del guest, traducendo gli indirizzi guest-virtual in host-physical (con un overhead non trascurabile).



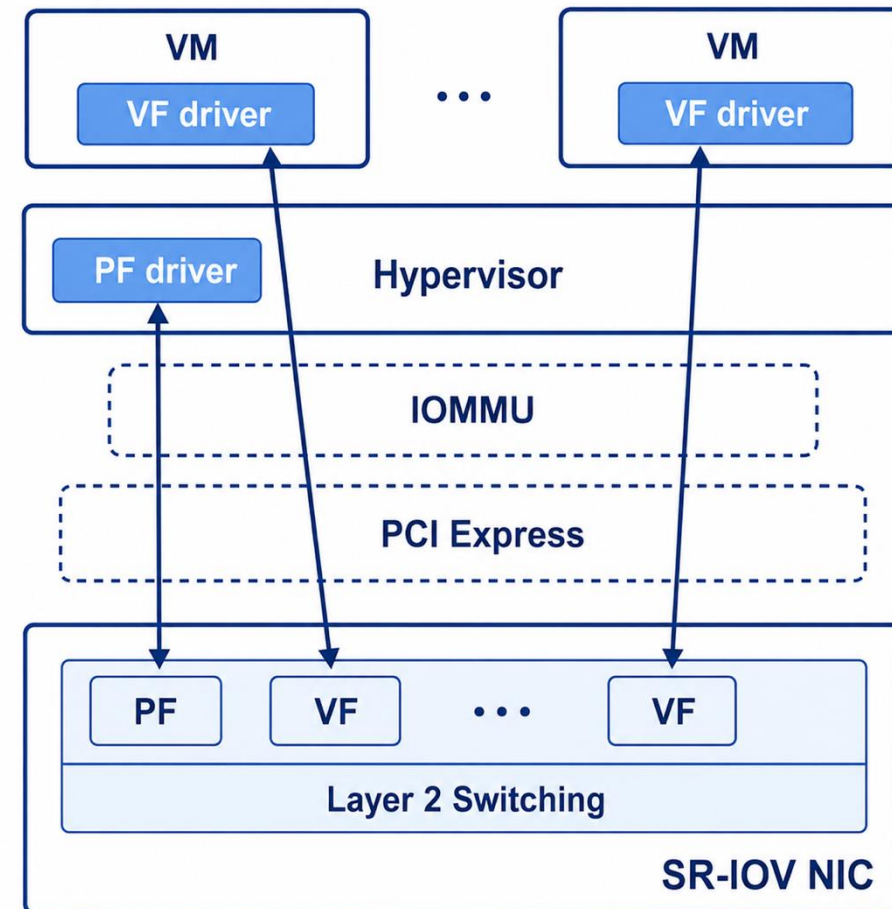
Virtualizzazione dell'I/O

Anche i dispositivi di I/O devono essere virtualizzati per i sistemi guest.

Emulazione del dispositivo: l'hypervisor emula via software il dispositivo hardware; soluzione molto portabile, ma lenta.

Driver paravirtualizzati (es. VirtIO): il guest OS dialoga direttamente con l'hypervisor attraverso interfacce ottimizzate, ottenendo prestazioni superiori.

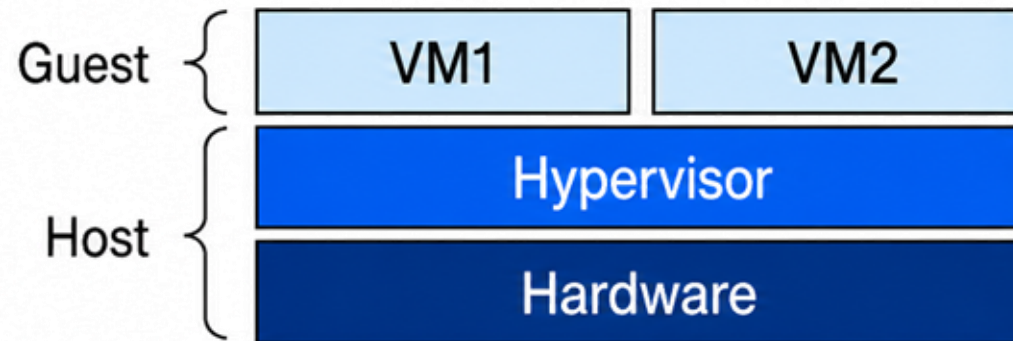
Assegnazione diretta del dispositivo: prestazioni vicine al nativo grazie al partizionamento dei dispositivi PCI, al prezzo di una minore flessibilità e di vincoli sulla migrazione.



Tassonomia degli hypervisor

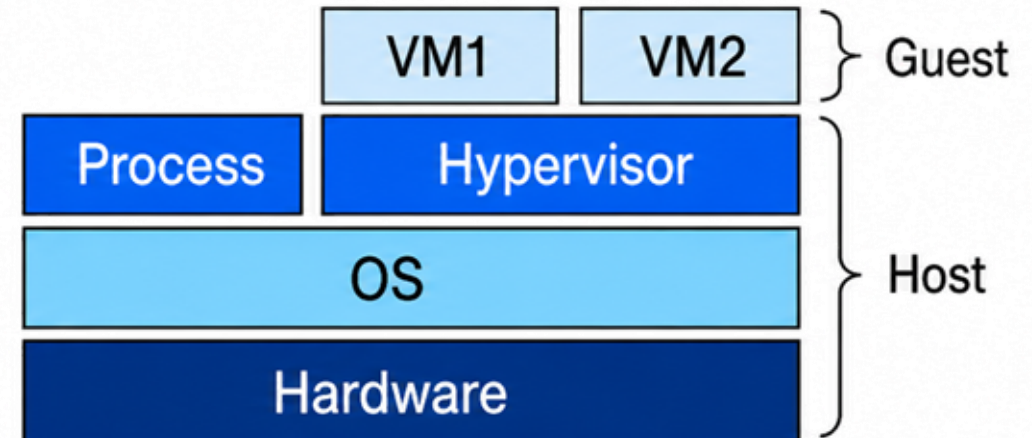
La tassonomia di *Goldberg (1973)* distingue gli hypervisor in base alla loro relazione con l'hardware: *Bare-metal* o *Hosted*.

Bare-Metal



VMware ESX, Microsoft Hyper-V,
Citrix XenServer

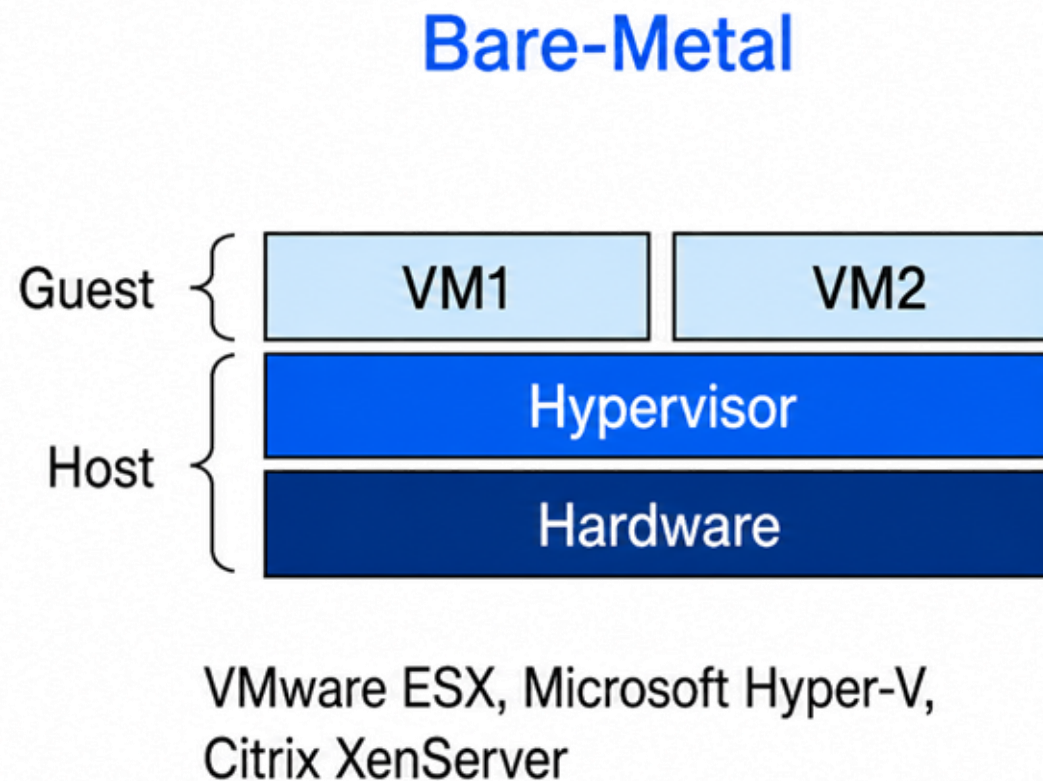
Hosted



VMware Workstation, Microsoft Virtual PC,
Sun VirtualBox, QEMU, KVM

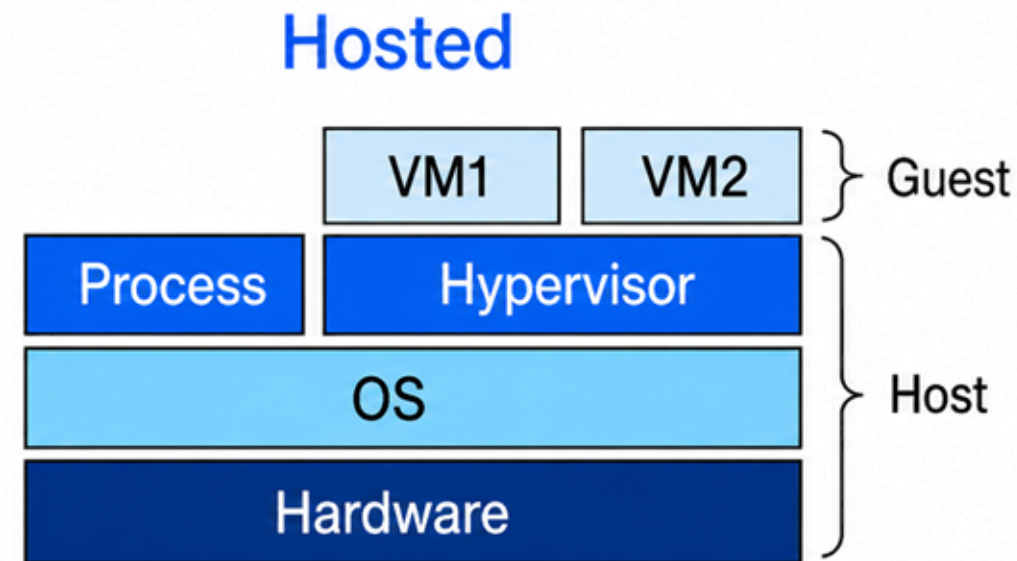
Architettura bare-metal

- In questa architettura l'hypervisor (o VMM) è installato direttamente sull'hardware nudo.
- L'hypervisor dialoga direttamente con l'hardware del sistema, senza appoggiarsi a un host OS.
- L'hypervisor è in pratica un OS leggero.
- Vantaggi
 - Migliori prestazioni di I/O.
- Svantaggi
 - Installazione e configurazione complesse.
 - Forte dipendenza dalla piattaforma hardware.



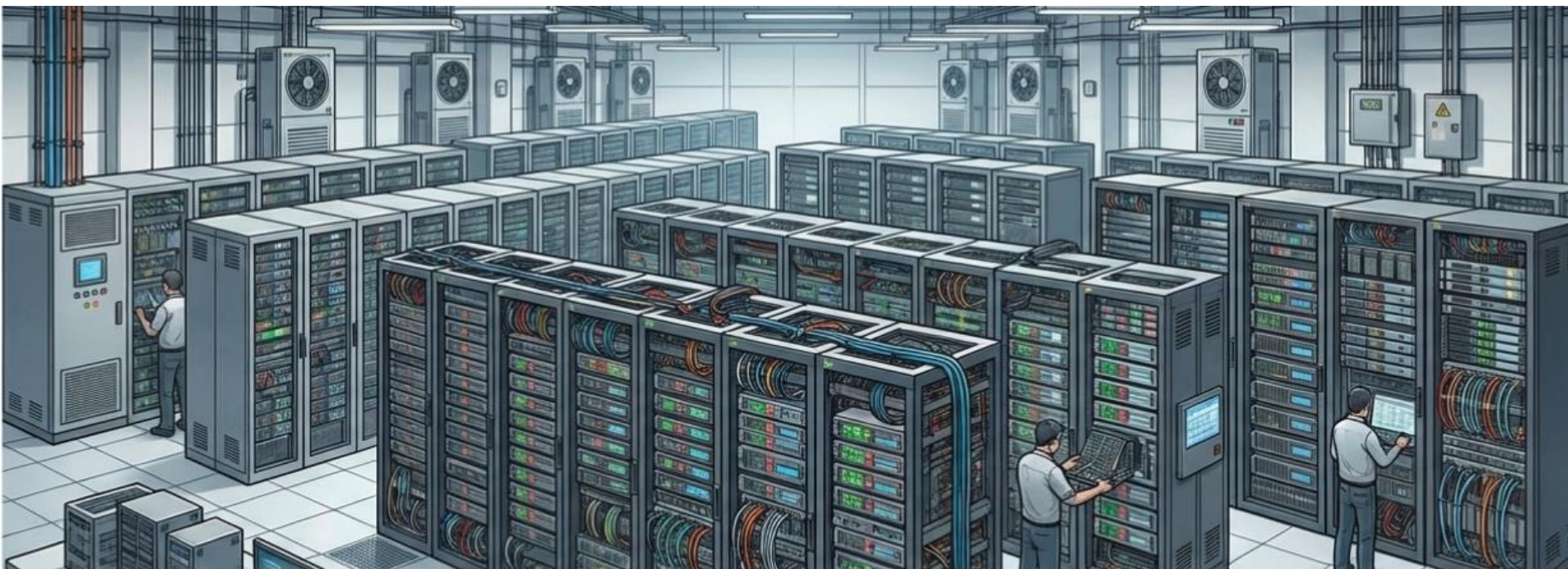
Architettura hosted

- Si installa per primo il sistema operativo host.
- L'hypervisor viene installato sopra il sistema operativo host.
- Permette agli utenti di eseguire diversi sistemi operativi guest in parallelo alle proprie applicazioni.
- **Vantaggi**
 - Installazione e configurazione semplici.
 - Host OS e guest OS non modificati.
 - Esecuzione su un'ampia gamma di computer.
- **Svantaggi**
 - Decadimento delle prestazioni.



VMware Workstation, Microsoft Virtual PC,
Sun VirtualBox, QEMU, KVM

Data center

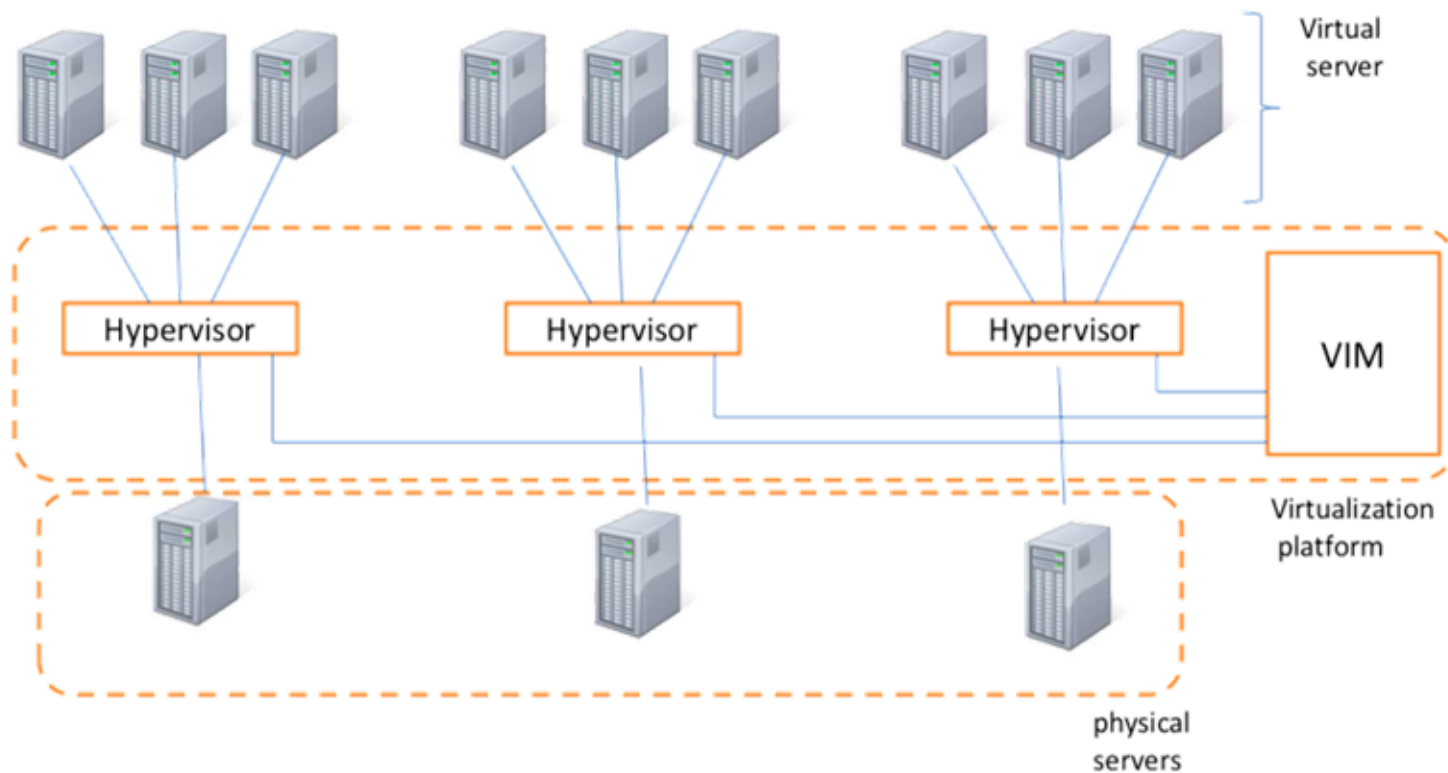


I data center sono edifici che ospitano centinaia di migliaia di server che, contemporaneamente, supportano molti servizi e applicazioni.

- da 50.000 a 80.000 server (da 25 a 30 MW)
 - spesso si utilizzano processori ARM per ridurre i costi energetici
- I data center sono costruiti su larga scala per ridurre i costi di manutenzione, ma non troppo grandi
 - anche per limitare il blast radius (raggio dei danni in caso di guasto)
- I data center sfruttano la virtualizzazione in modo massiccio per eseguire le applicazioni

Virtual Infrastructure Manager (VIM)

- Una **sfida** centrale per i cloud provider è **gestire risorse fisiche e virtuali** — server, storage e reti — in modo **unitario**.
- L'**orchestrazione delle risorse** deve essere effettuata in modo da fornire risorse a utenti e applicazioni in modo rapido e dinamico.
- Il software che si occupa di questa orchestrazione si chiama **Virtual Infrastructure Manager (VIM)**.
- Questo tipo di software assomiglia a hypervisor tradizionale, ma invece di gestire un singolo computer aggrega le risorse di più macchine e ne presenta una vista uniforme a utenti e applicazioni.



GESTIONE DELLE VM

Perché virtualizzare

La virtualizzazione è anzitutto uno strumento di *economia delle risorse* e di *disaccoppiamento* fra software e hardware sottostante.

01 · CONSOLIDAMENTO

Più carichi su un nodo

I server x86 dedicati sono utilizzati in media al **5–15 %**. Il consolidamento alza l'utilizzo a **60–80 %**, con riduzione di *capex*, energia, raffreddamento e spazio fisico.

02 · ISOLAMENTO

Confinamento dei guasti

Crash, panic e exploit di un guest non si propagano agli altri. Il VMM fornisce un *fault domain* più stretto del processo Unix tradizionale.

03 · MULTI-TENANCY

Isolamento

Tenant mutuamente non fidati condividono lo stesso hardware fisico.

04 · MOBILITÀ

Disaccoppiamento HW/SW

Snapshot, clonazione, migrazione live: una VM è un file portabile. Manutenzione hardware senza downtime applicativo.

05 · COMPATIBILITÀ

Sistemi legacy

Esecuzione di OS obsoleti su hardware moderno; ambienti di sviluppo riproducibili; emulazione di ISA estranee (es. ARM su x86).

Snapshot, checkpoint, Fault Tolerance

Idea di base

- La VM è un file: lo stato si può congelare, replicare, ripetere. Tre primitive ne reggono l'operatività in cluster.

Snapshot — stato istantaneo

- Cattura sincrona di memoria, registri, disco; il disco usa copy-on-write. Rollback in meno di un secondo, branching dell'esecuzione.

Checkpoint — snapshot persistente

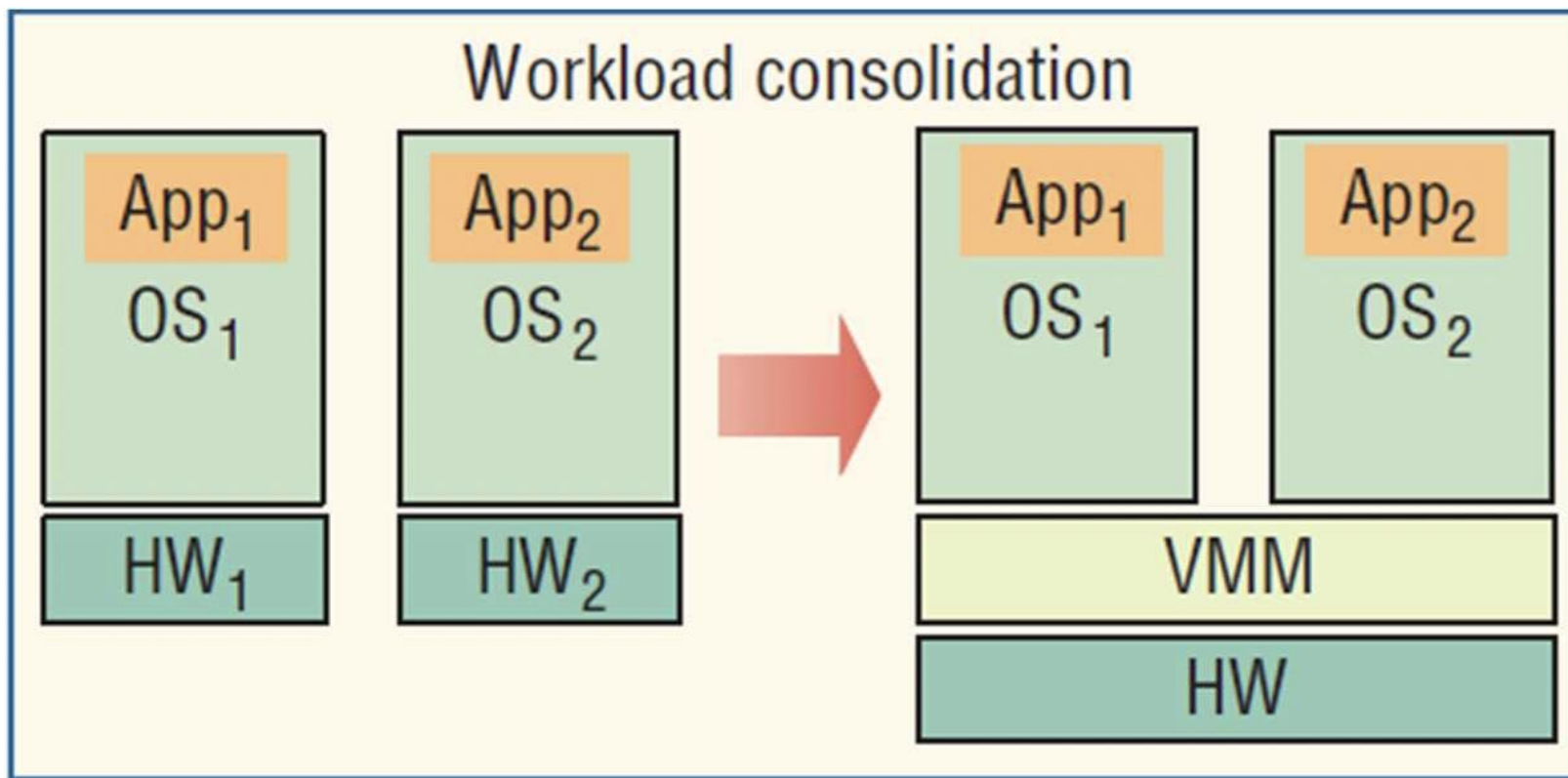
- Snapshot serializzato su storage durevole, con stato di rete e device. Abilita resume dopo spegnimento, debug record-and-replay, "criogenia" delle VM.

Fault Tolerance — replica continua

- VM secondaria mantenuta sincronizzata (lockstep o checkpoint ad alta frequenza). Se cade il primario, il secondario subentra senza perdita di stato osservabile.

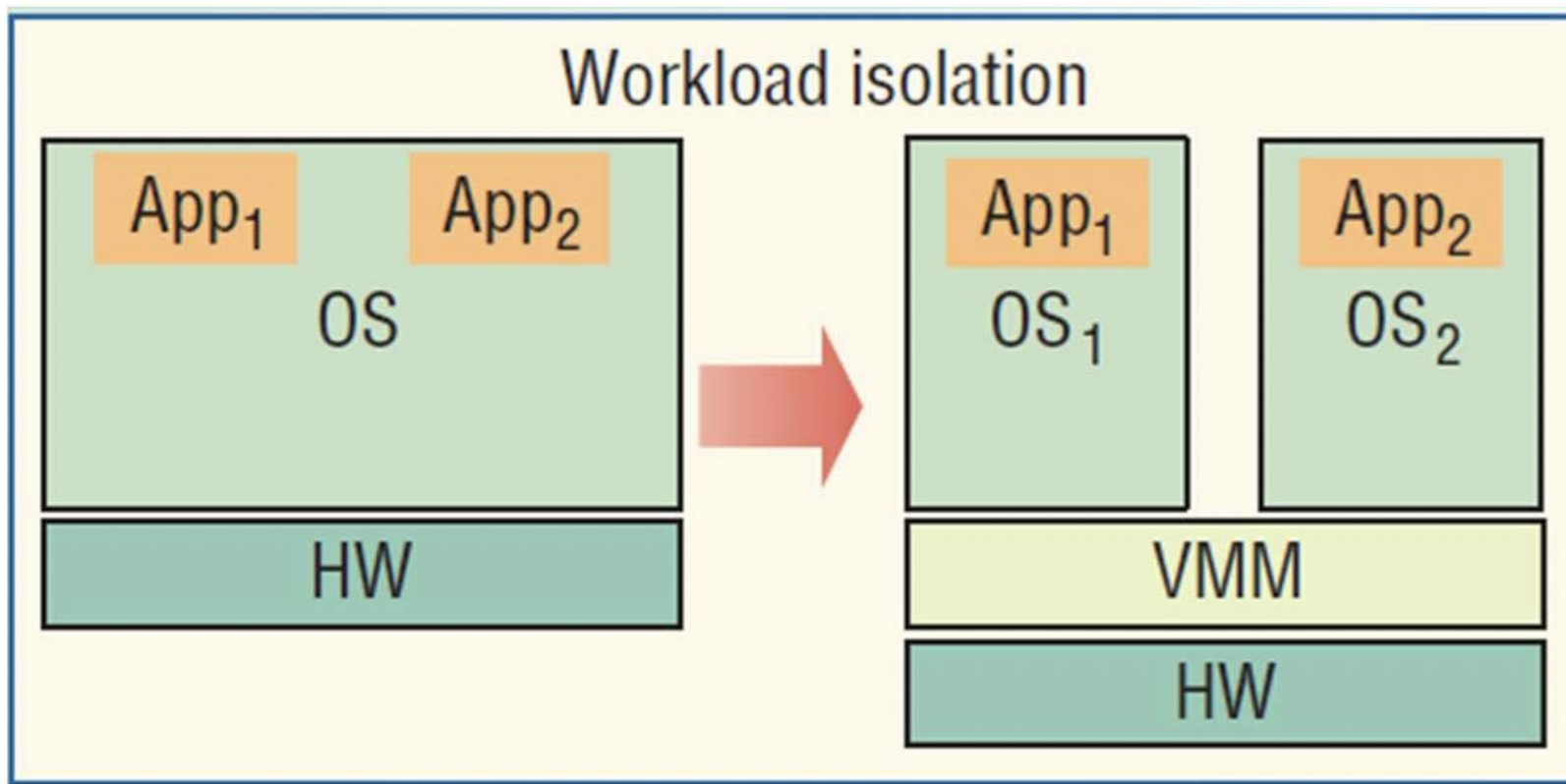
Consolidamento e multi-tenancy

- La virtualizzazione permette di consolidare più macchine virtuali su un singolo sistema fisico, riducendo il [total cost of ownership](#) (TCO).
- TCO: stima di tutti i costi diretti e indiretti legati all'acquisto e all'esercizio di un prodotto o sistema lungo l'intero ciclo di vita.



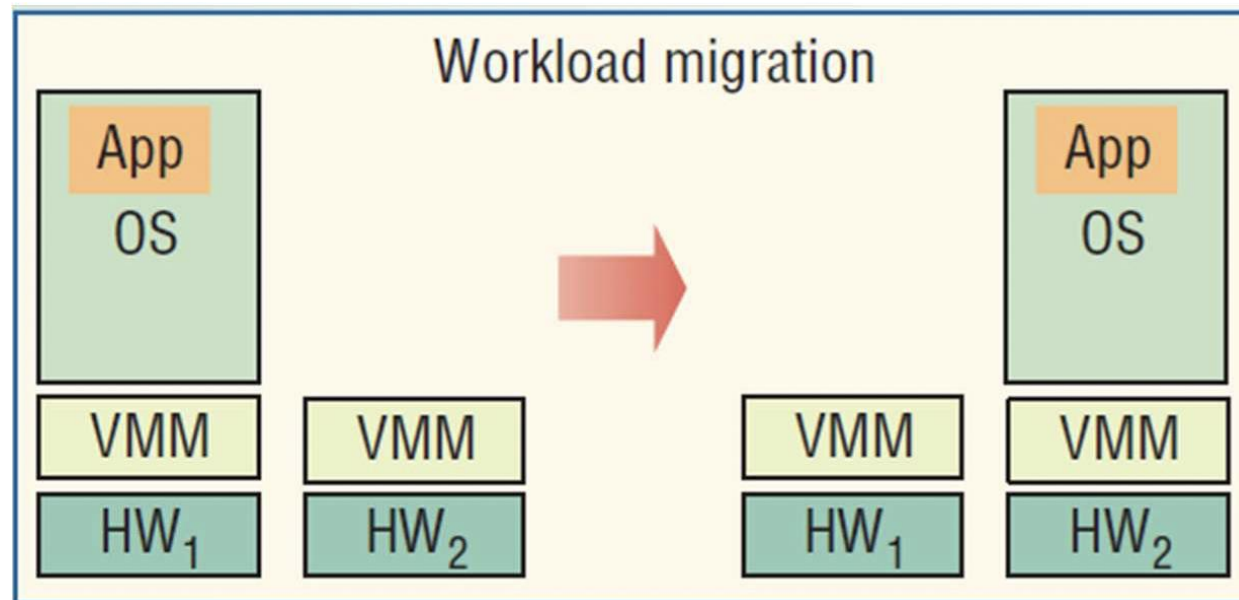
Isolamento dei carichi di lavoro

- Grazie alla virtualizzazione si ottiene l'isolamento del carico di lavoro: tutte le istruzioni di un programma restano confinate all'interno della VM, con benefici diretti sulla sicurezza.
- Affidabilità: i guasti software all'interno di una VM non si propagano alle altre.
- Controllo delle prestazioni: l'esecuzione di una VM non deve incidere sulle prestazioni di un'altra VM.



Migrazione dei carichi di lavoro

- La migrazione del carico di lavoro, detta anche application mobility, migliora:
 - la manutenzione hardware,
 - le prestazioni,
 - la fault tolerance.
- Si realizza permettendo al guest OS di una VM di essere sospeso, migrato su un'altra piattaforma e ripreso immediatamente, oppure conservato per essere ripristinato in un secondo momento.
- Lo stato di una VM include il disco completo, i file di configurazione e l'immagine della memoria (RAM).



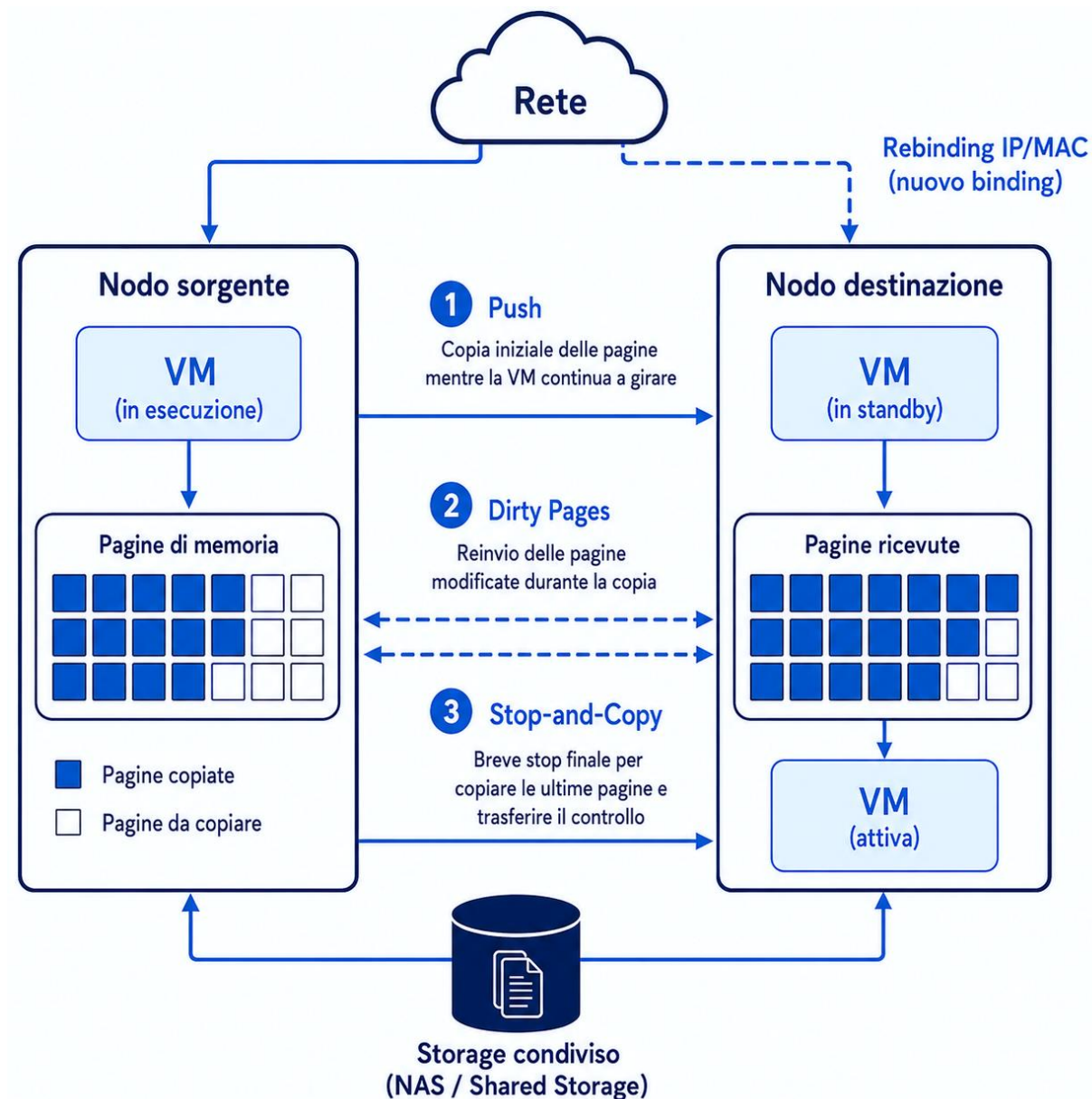
Mobilità forte – Strategia di Migrazione VM

Strategia Pre-copy (Clark et al., NSDI 2005)

- **Fase 1 (Push):** copia delle pagine di memoria verso la destinazione mentre la VM continua a girare.
- **Fase 2 (Dirty Pages):** reinvio delle pagine modificate durante la copia iniziale.
- **Fase 3 (Stop-and-Copy):** breve interruzione (downtime) per copiare le ultime pagine residue e trasferire il controllo.
- **Risultato:** tempi di inattività minimi (spesso < 4 secondi).

Migrazione delle risorse locali

- **Networking:** annuncio del nuovo binding indirizzo–MAC (in LAN) o tunneling/mobile IP (in WAN) per mantenere attive le connessioni.
- **Storage:** tipicamente risolto separando lo storage in un livello dedicato (shared storage tier). La migrazione richiede solo di ristabilire la connessione di rete verso il server di storage, senza trasferire fisicamente i file.



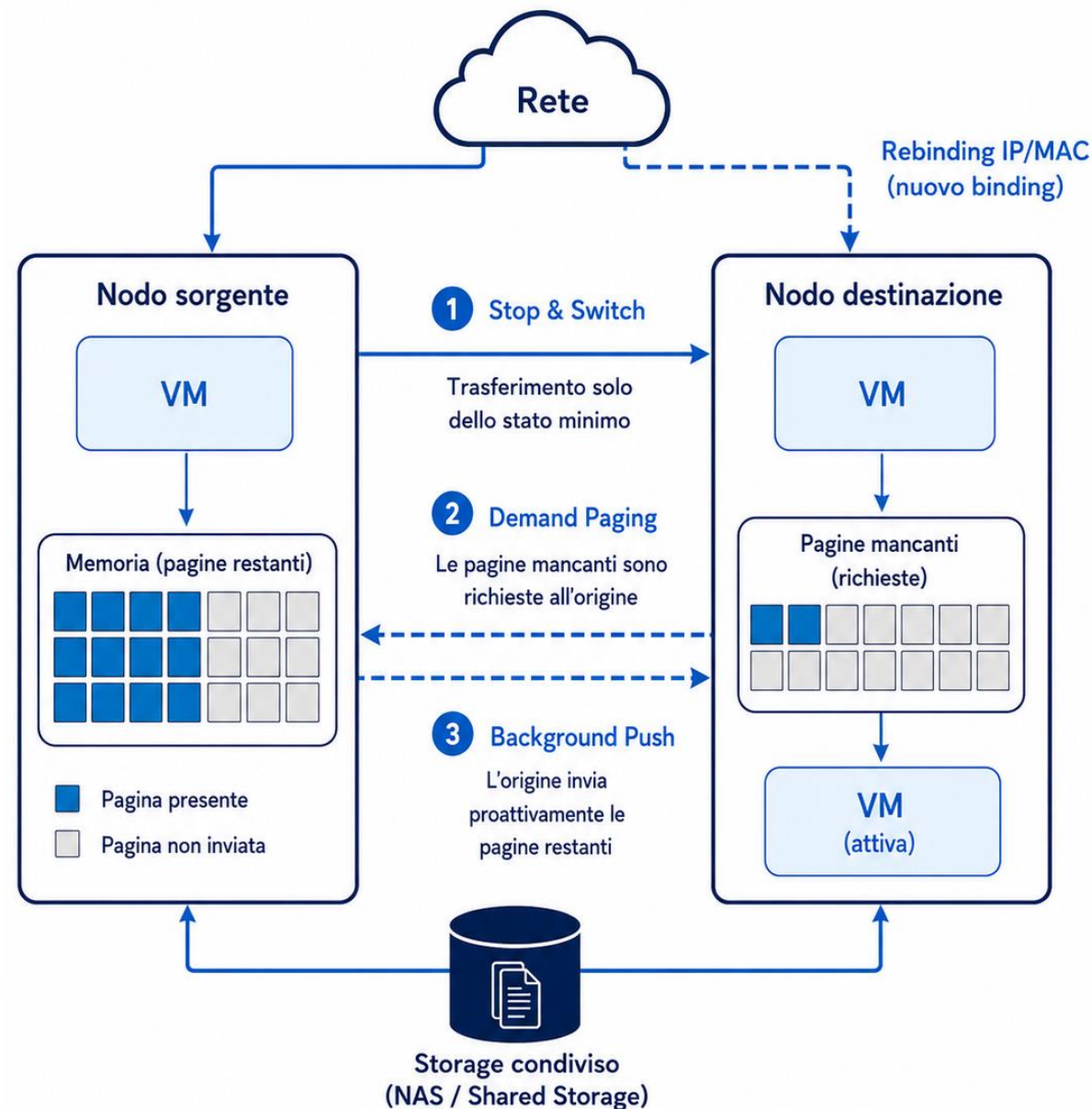
Mobilità forte – Strategia di Migrazione VM

Strategia Post-copy (Hines & Gopalan, VEE 2009)

- **Fase 1 (Stop & Switch):** la VM viene sospesa e si trasferisce subito solo lo stato minimo (CPU, registri, device).
- **Fase 2 (Demand Paging):** la VM riparte sulla destinazione; le pagine mancanti vengono richieste all'origine via rete (page fault remoti).
- **Fase 3 (Background Push):** in parallelo l'origine spinge proattivamente le pagine restanti per ridurre i fault.
- **Risultato:** downtime molto breve, ma performance degradate finché il trasferimento non è completo.

Migrazione delle risorse locali

- **Networking:** annuncio del nuovo binding indirizzo–MAC (in LAN) o tunneling/mobile IP (in WAN) per mantenere attive le connessioni.
- **Storage:** tipicamente risolto separando lo storage in un livello dedicato (shared storage tier). La migrazione richiede solo di ristabilire la connessione di rete verso il server di storage, senza trasferire fisicamente i file.



Consolidamento dinamico

La live migration permette di riorganizzare le VM tra gli host mentre tutto è in esecuzione: si possono accendere e spegnere server a seconda del carico, senza fermare i servizi.

Obiettivo

Concentrare le VM attive sul minor numero possibile di host, così quelli vuoti possono essere spenti per risparmiare energia.

Meccanismo

Lo scheduler del cluster controlla periodicamente il carico e, se è opportuno, sposta a caldo alcune VM verso altri host.

Quando scatta

Quando un host è sotto-utilizzato (si svuota e si spegne) oppure sovraccarico (alcune VM vengono spostate altrove).

Benefici Meno energia consumata • uso più efficiente dell'hardware • nessuna interruzione per gli utenti

Trade-off: il consolidamento aggressivo aumenta il rischio di colli di bottiglia se il carico cresce all'improvviso.